

AI Coding Agent Brief — FreelanceFlow CRM (MVP Build)

Paste this entire document as your first prompt in the AI IDE. Treat it as your full spec.

Do not wait for further instructions between sections — read everything once, then start executing Sprint 0.

1. Project Context (read this first, understand the "why")

Product: FreelanceFlow CRM — a lightweight CRM built specifically for freelancers and solo consultants.

Problem it solves:

- Freelancers lose leads because there's no central place to track them.
- Client communication is scattered across email, WhatsApp, and notes apps.
- Client retention is weak because there's no visibility into which clients are at risk of churning.
- Proposals are written manually every time, which is slow and inconsistent.

Vision: One dashboard where a freelancer can capture a lead, move it through a pipeline, convert it into a client and

project, generate an invoice, and get AI help drafting proposals and spotting at-risk clients — without needing five different tools.

Target user: An independent freelancer or small consultancy (1–5 people) juggling multiple clients, not a large sales team. Design every screen for one person moving fast, not an enterprise sales org.

Core value proposition: Fewer lost leads, faster proposals, and an early-warning signal on client health — in a tool that feels calm and professional, not cluttered.

You are building the **MVP only** — a working, demoable product that proves this value proposition end-to-end. Not the full 22-section platform described in the source document (that includes Kubernetes, microservices, enterprise tiers, mobile apps, and multi-channel integrations — none of that belongs in this build).

2. MVP Objectives

1. A freelancer can register, log in, and land on a dashboard.
2. They can add leads/clients and move them through a simple pipeline (New → Contacted → Proposal Sent → Won/Lost).
3. Won clients can have projects attached, with status tracking.
4. Projects can generate simple invoices (PDF), markable as paid/unpaid.

5. An AI assistant can (a) draft a proposal from client/project context and (b) compute a basic "client health score" from activity data.
 6. A dashboard shows pipeline value, revenue, and at-risk clients at a glance.
 7. The product must look like a deliberately designed SaaS tool, not a scaffolded demo.
-

3. Scope

In scope for MVP

- Email/password auth (+ Google OAuth if time allows)
- Leads/Clients CRM with pipeline stages
- Projects linked to clients
- Simple invoicing with PDF export (no live payment gateway — "mark as paid" manually is enough)
- AI proposal generation (from a form + client/project data)
- AI client health scoring (rule-based + LLM-assisted summary, not a trained ML model)
- Dashboard with basic charts (pipeline value, revenue this month, at-risk clients)
- In-app notifications (e.g. "Proposal not followed up in 5 days")
- Responsive layout (desktop-first, usable on tablet/mobile)

Explicitly out of scope for MVP (do not build these)

- Mobile app, WhatsApp/LinkedIn integrations
- Microservices, Kubernetes, multi-cloud, Terraform
- Meeting transcription/summarization (needs audio pipeline — stretch goal only if everything else is done and stable)
- Forecasting models beyond a simple trend line
- Multi-tenant/enterprise billing tiers, granular RBAC beyond Owner/Admin
- Real payment processing (Stripe live mode) — stub it, don't integrate it
- Internationalization/localization
- Disaster recovery, advanced monitoring stacks (Grafana/Prometheus)

If you're ever unsure whether something belongs in the MVP, default to leaving it out and note it in `docs/DECISIONS.md` as a future enhancement instead of asking me.

4. Tech Stack — mandatory, all free and open-source (or generous free-tier)

Layer	Choice	Why
Frontend framework	React 18 + Vite + TypeScript	Fast dev loop, free, huge ecosystem
Styling	Tailwind CSS + shadcn/ui	Consistent, accessible

Layer	Choice	Why
		components — prevents "vibe coded" inconsistency
Fonts	Inter (body) + Space Grotesk (headings), via Google Fonts, self-hosted with <code>@fontsource</code>	Both free, open-license, not the default system font stack
State/data fetching	TanStack Query + Zustand	Free, minimal boilerplate
Forms/validation	React Hook Form + Zod (shared schema client + server)	Free, type-safe
Charts	Recharts	Open source, matches shadcn aesthetic
PDF generation	<code>@react-pdf/renderer</code>	Open source, no external service needed
Backend	Node.js + Express + TypeScript	Simple modular monolith — no need for

Layer	Choice	Why
		microservices at MVP scale
ORM	Prisma	Open source, works cleanly with Postgres, great DX
Database	PostgreSQL via Supabase (free tier)	Free Postgres + Auth + Storage in one place, no 30-day expiry like some free DB hosts
Auth	Supabase Auth	Free, handles email/password + OAuth, saves you from building auth from scratch
AI / LLM	Groq API (free tier, e.g. llama-3.3-70b-versatile or llama-3.1-8b-instant)	No credit card required, very low latency, generous enough for MVP demo traffic
Local AI dev/testing	Ollama (optional, for offline iteration	Fully free, no API limits while

Layer	Choice	Why
	before wiring Groq)	developing prompts
File/object storage	Supabase Storage	Free tier, already bundled with the DB/auth choice
Frontend hosting	Vercel (Hobby/free tier)	Zero-config React deploys
Backend hosting	Render (free web service tier)	No credit card, 750 free instance hours/month — acceptable for an MVP demo (note: free tier sleeps after 15 min idle; document this limitation, don't try to work around it)
CI/CD	GitHub Actions	Free for this repo size
Testing	Vitest (unit) + Playwright (E2E) + Supertest (API)	All open source

Layer	Choice	Why
Error tracking	Sentry free tier	5k events/month is enough for an MVP
Version control	Git + GitHub	Free

Do not substitute paid services (e.g. no Auth0, no paid Stripe integration, no paid monitoring) anywhere in this build.

5. Architecture & Repository Structure

Modular monolith, not microservices. Keep it simple enough that one person can reason about the whole system.

```
freelanceflow/
├── frontend/           # React + Vite app
│   ├── src/
│   │   ├── components/ # shadcn-based UI components
│   │   │   ├── features/ # leads, clients, projects, invoices, ai, dashboard
│   │   │   ├── lib/
│   │   │   └── styles/
│   └── backend/        # Express + TypeScript API
│       ├── src/
│       │   ├── modules/ # auth, leads, clients, projects, invoices, ai, notifications
│       │   └── middleware/
```

```
| | └─ lib/
└─ database/
  └─ prisma/          # schema.prisma,
migrations
└─ docs/
  └─ DECISIONS.md     # every assumption
you make, logged here
  └─ README.md
└─ .github/workflows/ # CI pipeline
└─ tests/
```

6. Design Requirements — this must NOT look AI-generated or "vibe coded"

Follow these rules strictly on every screen:

- **Typography:** Use the Inter/Space Grotesk pairing defined above. No default system fonts. Headings get tighter letter-spacing (-0.01em to -0.02em); body text stays at normal spacing with 1.5–1.6 line-height.
- **Color system:** One primary color (pick a restrained slate/indigo, not a bright purple-to-pink gradient), a neutral gray scale for backgrounds/borders/text, and one accent color for success/warning/danger states each. No rainbow gradients, no glassmorphism clichés.
- **No emojis anywhere** — not in UI copy, not in button labels, not in empty states, not in code comments, not in commit messages, not in console logs.
- **Spacing:** Use a consistent 4px/8px spacing scale throughout (Tailwind's default scale is fine — just be consistent, don't mix arbitrary values).

- **Components:** Build once, reuse everywhere via shadcn/ui primitives. Buttons, inputs, cards, and modals should look identical in every module.
 - **States that matter:** Every list/table needs a real empty state (not "No data"), a loading skeleton (not a spinner-only screen), and an error state with a retry action.
 - **Restraint:** Avoid `rounded-full` buttons everywhere, avoid stacking multiple drop shadows, avoid decorative gradients on cards. Flat, clean, confident — like Linear or Notion, not like a hackathon template.
-

7. Functional Requirements (MVP detail)

1. **Auth** — sign up, log in, log out, password reset. Session persisted via Supabase Auth.
2. **Leads/Clients CRM** — create/edit/delete a lead, pipeline board (drag-and-drop or simple dropdown to change stage), convert a "Won" lead into a client record.
3. **Projects** — create a project under a client, set status (Not started / In progress / Completed), basic timeline fields (start date, due date).
4. **Invoices** — create an invoice against a project (line items, amount, due date), generate a PDF, mark as Paid/Unpaid/Overdue.
5. **AI Assistant:**
 - *Proposal generator:* given client name, project scope, and rough budget/timeline typed by the user, call the LLM to draft a proposal the user can edit before sending.

- *Client health score*: compute a score from simple signals you have in the DB (days since last contact, number of open projects, overdue invoices, pipeline stage stagnation) and use the LLM to turn that into a one-line human-readable summary (e.g. "At risk – no contact in 18 days, one overdue invoice").
6. **Dashboard** – total pipeline value, revenue this month (from paid invoices), a short list of at-risk clients (from the health score), and a simple trend chart.
 7. **Notifications** – in-app only for MVP (a notification bell with a list), triggered by: proposal sent but no follow-up in N days, invoice overdue, client health score dropped to "at risk."
-

8. Data Model (high level – you own the exact schema)

Core entities: `User` , `Client` , `Lead` (or a `status` field on `Client` if you prefer to merge them), `Project` , `Invoice` , `InvoiceLineItem` , `Notification` , `ActivityLog` (used to compute health score – log contact events, stage changes, etc.).

Design the Prisma schema yourself based on the functional requirements above. Keep relationships normalized and add the indexes you'd actually need for the dashboard queries (e.g. on `client.status` , `invoice.status` , `invoice.dueDate`).

9. Non-negotiable Engineering Rules

- TypeScript strict mode on both frontend and backend. No `any` unless truly unavoidable, and comment why.
 - Validate input on both client (Zod + React Hook Form) and server (same Zod schemas, shared via a common package if convenient).
 - Centralized error handling middleware on the backend — no unhandled promise rejections.
 - Secrets only via `.env`, never committed. Provide a `.env.example` with placeholder values.
 - Conventional commits (`feat:`, `fix:`, `chore:`, etc.), no emojis, no filler messages like "update stuff."
 - Every module (leads, projects, invoices, ai, auth) needs at least basic unit tests for its core logic, and the critical user flows (signup → add lead → convert → add project → invoice → mark paid) need one Playwright E2E test.
 - Lint and type-check must pass with zero errors before a sprint is considered done.
-

10. Execution Plan — Sprints and Checkpoints

Work through these sprints **in order, autonomously**. At the end of each sprint, run the self-verification checklist below before moving to the next one.

Sprint 0 — Scaffolding

Set up the monorepo structure, Vite + Express boilerplate,

Tailwind + shadcn install, Prisma connected to Supabase, CI pipeline skeleton, `.env.example`, base design tokens (fonts, colors) wired in. Deploy a "hello world" version to Vercel/Render to confirm the pipeline works end-to-end before building features.

Sprint 1 – Auth + Core CRM

Supabase Auth integration (signup/login/logout/reset), protected routes, Leads/Clients CRUD, pipeline stage UI.

Sprint 2 – Projects + Invoices

Project CRUD linked to clients, invoice creation + PDF export, paid/unpaid status handling.

Sprint 3 – AI Assistant

Wire up the Groq API, build the proposal generator flow, build the client health score computation + LLM summary. Handle API failures gracefully (loading states, retries, fallback copy if the LLM call fails).

Sprint 4 – Dashboard + Notifications

Build the dashboard charts and at-risk client list, implement in-app notifications and their triggers.

Sprint 5 – Hardening, Testing, Polish, Deployment

Fill test coverage gaps, run the full E2E flow, fix every console warning and lint error, do a full design pass against Section 6, deploy the final build, and write `docs/README.md` with setup/run instructions.

Self-verification checklist (run after every sprint, before moving on)

- ☐ `npm run build` succeeds on both frontend and backend with zero errors
- ☐ `npm run lint` and type-check pass with zero errors
- ☐ Relevant unit/E2E tests for this sprint's features pass
- ☐ Manually reasoned through the sprint's user flow end-to-end (describe this reasoning briefly in your own log, don't skip it)
- ☐ Any assumption made this sprint is logged in `docs/DECISIONS.md`
- ☐ No emojis, no unstyled default-HTML elements, no leftover placeholder text anywhere touched this sprint

If a check fails, fix it before proceeding — do not carry broken checkpoints into the next sprint.

11. Autonomy Instructions

- Do not pause mid-build to ask me clarifying questions. If something is ambiguous, make the most reasonable product decision yourself, write it down in `docs/DECISIONS.md` with a one-line rationale, and keep going.
- After finishing each sprint's checklist, proceed automatically to the next sprint. Do not stop for my confirmation between sprints.
- Debug and fix issues yourself as you find them. Only bring something to me if it is a genuine external blocker (e.g. a required paid API key, an account you don't have access to) — not a design or implementation choice.

- The only point at which you should stop and report back to me is after Sprint 5 is fully complete, the app is deployed, and it has been debugged to as close to zero errors as reasonably achievable.
-

12. Definition of Done

- All MVP functional requirements in Section 7 are implemented and working.
 - Build, lint, type-check, and test suite all pass cleanly.
 - The app is deployed and reachable at a live URL (frontend + backend).
 - The UI follows the design requirements in Section 6 — no emoji, no default fonts, consistent spacing/components.
 - `docs/DECISIONS.md` documents every assumption made along the way.
 - `docs/README.md` explains how to run the project locally and what environment variables are needed.
-

13. Final Report Expected From You

Once Definition of Done is met, stop and give me:

1. A summary of what was built, sprint by sprint.
2. The live URL(s) for frontend and backend.
3. Any known limitations or rough edges, and why they were left for later.

4. The full list of assumptions from `docs/DECISIONS.md` .
5. Local setup instructions.

Do not ask me anything before that point. Start with Sprint 0 now.